

Distributed Banking Systems and Payment Architecture

Reference for microservices, ledger, concurrency and distributed reliability.

Microservices

Each service owns a bounded business capability and its data. BankFlow separates authentication, users, accounts, ledger, transactions, payments, beneficiaries, banks, settlement, reconciliation and notifications.

Services communicate through APIs and asynchronous events rather than querying another service's database.

ACID Transactions

Atomicity means a financial operation commits completely or not at all.

Consistency preserves database constraints and business invariants.

Isolation controls concurrent interaction.

Durability means committed data survives normal failures.

Concurrency

PostgreSQL row-level locking such as `SELECT ... FOR UPDATE` can serialize modifications to the same account state.

Optimistic locking can use a version number to detect concurrent modifications.

Isolation levels include `READ COMMITTED`, `REPEATABLE READ` and `SERIALIZABLE`.

Double-Entry Ledger

Every balanced journal has equal total debits and credits.

The ledger provides an auditable accounting trail; balance projections must remain consistent with it.

Event-Driven Architecture

Kafka can carry durable domain events such as payment completion, ledger posting, settlement and reconciliation.

Consumers should be idempotent because delivery can repeat.

RabbitMQ can handle background work such as notifications and statement generation.

Transactional Outbox

A service writes its database state change and an outbox event in one transaction. A publisher later delivers the event to Kafka.

This prevents a successful database commit from being separated from event publication.

Saga and Compensation

A distributed transaction can be decomposed into local transactions coordinated by a Saga.

If a later step fails, compensation corrects earlier effects. Compensation preserves history rather than deleting financial records.

Caching and Locks

Redis can provide caching, rate limiting, sessions and selected distributed coordination.

Redis is not the financial source of truth; PostgreSQL and the ledger are authoritative.

Consistency

Financial posting should be strongly consistent within its transaction boundary.

Notifications, analytics and read projections may be eventually consistent.

Failure Modes

Partial failure occurs when one service succeeds while another fails.

Timeout creates uncertainty about remote state.

Duplicate delivery requires idempotent consumers.

Reconciliation resolves disagreements between systems.